

Agentic Assort Planner — Resume & Interview Playbook

Senior SWE · Python + AI/Agents + Data · grounded in your own repos, PoC & audit — July 2026

This is a single place to (1) copy **resume-ready bullets** for the **Agentic Assort Planner** project, (2) understand the **detailed backing** for every claim — the *why*, the *tradeoffs*, the *decision path*, the *impact* — and (3) walk into an interview able to defend the **architecture, databases, AI/agent design, and every technology named**, down to fundamentals. Written for a **senior software engineer (Python + AI + data)**.

How to use this doc

- **§1** — the project one-liner + summary blurb (paste at the top of the project).
- **§2** — the **XYZ resume bullets** (pick 5–8), each followed by a **backing block** (how/why/tradeoff/impact/evidence/honesty).
- **§3** — **product architecture (HLD)** — the agentic planning workspace + agent orchestration.
- **§4** — **database architecture (HLD + LLD)** — three-tier spine, the tenant-unified derived schema, and the ClickHouse insert-only/versioned design.
- **§5** — **per-technology deep dives** — fundamentals · architecture · how it solves the problem · gotchas, for every tech you'd list.
- **§6** — **interview Q&A** — the questions an engineering manager / architect will ask, with model answers.
- **§7** — **talking tracks + honesty guardrail** — 30-sec / 2-min / deep pitches, STAR stories, and exactly which numbers are real vs indicative.
- **§8** — **authoritative product specifics** from the requirements notebook (stack, autonomy bands, ClickHouse PoC numbers).
- **§9** — **the Cluster Recommendation Copilot** — the agentic clustering module now in build (FRD v1.8): what/why/how, measured baselines, targets, and its own Q&A. This is your *present-tense* flagship — the answer to "what are you working on right now?"

The format: Google's XYZ formula

Every strong bullet follows "**Accomplished [X], as measured by [Y], by doing [Z]**" — X = the outcome, Y = the metric, Z = the method/tech. Lead with impact, quantify it, then name the technique. For a *senior* role the narrative is "**I architected systems and multiplied team output,**" not "I wrote code" — so bullets emphasize design decisions, cross-system tradeoffs, cost/latency wins, and de-risking. Structure the résumé as a 2–3 line summary, then 5–8 quantified XYZ bullets per project. (Sources: Wonsulting, Teal, IGotAnOffer — linked at the end.)

Honesty guardrail — read before you paste. Keep only what **you personally** did; this doc is scoped to work evidenced in your repos/PoC/audit, but you must still own each line in the room. Two tiers of numbers appear here:

- **REAL (defend as measured):** the BigQuery cost figures ($\approx 3\times / 74\times / 280\times / 880\times$; the 38.8 TiB / \$327 `SELECT *` leak), the Postgres audit facts (88% orphaned optimizer rows; 5,503 partitions for 298K rows), and the catalog verification (5,385 columns, 46 FKs) — these come from the **live mtp audit + read-only metadata**.
- **INDICATIVE (frame as "in an offline eval" / "design target"):** the decomposition score 0.54→0.99, and the PoC latency/seed/pivot milliseconds — these are **mock / pure-Python** harness results measuring *structure & correctness*, not production model accuracy or real-engine latency. The load-bearing PoC result that IS real is the **architecture property**: `mutations_used = 0` at every scale.
- **FRD (Cluster Copilot, §9):** two sub-tiers — **measured baselines** from live tenant audits (8.5% run failures = 37/437; 0% reproducibility; median job ≈ 20 s over 370 runs; 1-in-5 finalized plans with zero stores) which you defend as "measured on live tenants," and **committed targets** (<1 h to finalized plan, ≥ 20 configs, <2% failures, p95 <500 ms) which you frame as "the design targets we set and are building against."

Say "in an offline evaluation" or "modeled/projected" for the indicative ones and you'll never be caught out.

1. Project summary (paste-ready)

One-liner (résumé project header):

Agentic Assort Planner — AI-assisted, multi-agent retail merchandise-planning platform (Impact Analytics). *Python · LLM agents (OpenAI/Gemini, LangGraph-style orchestration) · BigQuery + Postgres + ClickHouse/DuckDB · FastAPI.*

2–3 line summary (top of the project block):

Designed and built the AI and data backbone of a next-generation assortment-planning product: a **router→orchestrator→specialist multi-agent system** that turns natural-language planning questions into a validated execution DAG, and a **three-tier data architecture** (BigQuery source-of-truth → Postgres

editable system-of-record → ClickHouse/DuckDB OLAP serving) with a **tenant-unified schema at the derived grain**. Currently building its first shipping module — an **agentic store-clustering copilot** (deterministic tools on a dedicated ClickHouse read plane, LLM orchestration, three human confirm-gates) targeting <1 h to a finalized plan (from days), ≥20 configurations explored (from 1) and <2% run failures (from a measured 8.5%). Drove the ClickHouse decision via a runnable PoC and a cost/perf audit that cut recurring BigQuery scan cost by up to ~280x.

Current-work one-liner (for "what are you doing right now?"):

"I'm building the clustering module of the agentic planner — a copilot where the planner states *what to cluster and for when*, and a dedicated agent handles store scoping, attribute selection and batch exploration on a dedicated ClickHouse read plane, with evidence packs and three human approval gates; approved clusters write back into the existing product unchanged. We spec'd it against live-audit baselines — 8.5% run failures, zero reproducibility, one configuration tried per plan — and the targets are <1 hour to a finalized plan, ≥20 configs explored, and p95 sub-500 ms agent data probes." (*Full depth: §9.*)

How to describe the product in one breath (for interviews):

"It's the agentic rebuild of our assortment planner. A merchant works a plan through a staged pipeline — hindsight → strategy → clustering → line plan → buy plan — and a fleet of specialist AI agents runs alongside: surfacing issues as *Signals*, proposing scenarios, and acting autonomously overnight within approval thresholds. Under the hood it's a multi-agent orchestrator plus a three-tier data spine designed so a pivot-heavy, editable planning grid and scan-heavy hindsight analytics each run on the engine that's best at them."

Where to place it on the résumé: flagship project under Impact Analytics. Lead the project with 2 AI/agentic bullets and 2 data-architecture bullets (the most senior, best-evidenced work), then 1–2 cost/perf or leadership bullets depending on the JD (§7 has tailoring guidance).

2. Resume bullets (XYZ) + detailed backing

Pick **5–8**. Each bullet is followed by a **backing block** — *why you did it, how, the tradeoff, the decision path, the impact, the evidence, and the honesty note* — so you can defend every word. Metrics tagged **[REAL]** are measured from the live audit/metadata; **[EVAL]** = offline/mock harness; **[DESIGN]** = design target / architecture

property; **[FRD]** = a measured baseline or committed target from the Agentic Cluster FRD v1.8 (say "measured baseline" for the former, "design target" for the latter).

2.1 Flagship — AI / agentic orchestration

A1 — Multi-agent decomposition

Architected a router→orchestrator→specialist multi-agent system (5 domain agents) that decomposes a natural-language planning question into a validated execution DAG, raising decomposition quality from 0.54 to 0.99 across a 12-query offline eval by adding a structural DAG validator plus a single LLM self-repair pass — removing the No. 1 production failure of LLM planners (malformed plans).

Backing. *Why:* free-form "let the LLM plan" produces invalid or under-specified plans that break downstream execution. *How:* a **Router** LLM classifies intent → `single | orchestrate` ; a **Decomposer** emits `{sub_queries[], path[][]}` (an agent-routed DAG); a `plan_schema.validate()` gate rejects unknown domains, duplicate ids, over-wide stages, and — critically — **forward references** in `<A_n>` placeholders before anything executes; on failure it re-prompts the LLM once with the validation errors (`repaired=True`). *Tradeoff:* one extra LLM round-trip on the ~small fraction of invalid plans vs. cascading executor failures — cheap insurance. *Decision path:* chose a **prompt + hard validator** over a fine-tuned planner (faster, deterministic, debuggable) and over free-form ReAct (unpredictable). *Impact:* structural correctness 0.54→0.99 **[EVAL, 12 gold queries]**; the validator makes failures explicit and recoverable. *Evidence:* `agentic_assort_multiagent/core/{router,decomposer,plan_schema,orchestrator}.py` ; `docs/03` . *Honesty:* the score is an **offline structural eval** on mock model outputs — say "in an offline evaluation harness."

A2 — Parallelism + dependency passing + caching

Cut agent-pipeline latency by executing independent sub-queries in parallel within each DAG stage (thread pool) and passing cross-agent results through `<A_n>` placeholder rephrasing, and eliminated repeat work with a pgvector semantic cache (cosine similarity + TTL) that short-circuits the whole pipeline on similar questions.

Backing. *Why:* a planning question fans out to several agents; running them serially is needlessly slow, and users re-ask near-identical questions. *How:* the orchestrator walks `plan.path` **stage-by-stage (sequential)** but runs agents **within a stage concurrently** via `ThreadPoolExecutor(max_workers=max_per_stage)` ; when a later stage depends on an earlier answer, a **rephraser** substitutes `<A1>` with the prior result before dispatch; a **SemanticCache** embeds the query and returns a cached answer when cosine-similarity ≥ threshold and TTL is fresh (pgvector `<=>` in prod, in-memory offline). *Tradeoff:* thread pool (not asyncio) because the agent I/O is

largely blocking DB/LLM SDK calls — simpler and correct for the workload; cache adds a staleness window bounded by TTL. *Impact*: removes serialization on the fan-out and avoids full-pipeline recompute on repeats. *Evidence*: `core/orchestrator.py` , `connectors/cache.py` . *Honesty*: design/architecture property; no production latency number attached.

A3 — Resilient agents + safe text-to-SQL

Built self-contained specialist agents on a `validate→fetch→analyze→narrate` lifecycle with exponential-backoff retries and partial-failure synthesis (the orchestrator still returns a useful answer when one agent errors), and generated text-to-SQL with hard guardrails — `SELECT`-only, `LIMIT` caps, `NULLIF` denominators, no `SELECT *` .

Backing. *Why*: one flaky data call shouldn't fail the whole answer, and LLM-authored SQL is a foot-gun (runaway scans, division-by-zero, full-table reads). *How*: `BaseAgent.answer()` never raises — a failure returns `AgentResult(status="error")` so `_synthesize()` produces a **PARTIAL** answer; `with_retries()` wraps transient `fetch()` with 0.2/0.4/0.8s backoff; the text-to-SQL prompt is constrained and the result is validated before execution. *Tradeoff*: partial answers must be clearly labeled to the user (trust) — surfaced via status + reason. *Impact*: graceful degradation instead of hard failure; bounded, safe queries. *Evidence*: `agents/base.py` , `core/retry.py` . *Honesty*: implemented in the PoC; production hardening (timeouts, circuit-breakers) is the next step — good thing to volunteer.

A4 — Governed autonomy (human-in-the-loop)

Designed the agent autonomy model — agents act unattended only within confidence/impact thresholds; above threshold they raise a human-in-the-loop "Signal" with a reason code, and every agent action and human decision is written to an immutable audit log — enabling safe overnight automation.

Backing. *Why*: merchants won't trust an agent that silently changes a multi-million-dollar buy; autonomy must be bounded and auditable. *How*: `isAutonomous` / `aboveThreshold` gate actions; the **Signals** worklist carries Accept/Skip/Edit/Dismiss + reason; the **Log/audit trail** records who/what/when; overnight actions produce a morning summary. *Tradeoff*: thresholds too tight → agents feel useless; too loose → risk. Chose explicit, per-signal confirmation above threshold + full audit as the trust anchor. *Impact*: makes "overnight autonomy" acceptable to the business. *Evidence*: `product_artifacts/...Knowledge.md` (Signals/Log/autonomy threshold). *Honesty*: product-design + prototype behavior; frame as "designed the governance model."

2.2 Flagship — Agentic Cluster Recommendation Copilot (the module now in build)

This is the concrete, shipping instantiation of the agentic vision: a dedicated agent for the store-clustering module, spec'd in the Agentic Cluster FRD v1.8 (companion HLD/LLD) with measured baselines from live tenant audits. These bullets are the freshest and most defensible "senior owns a system end-to-end" material — lead with 1–2 of them.

C1 — The copilot itself (own the system story)

Designed an agentic store-clustering copilot — **deterministic tools compute on a dedicated ClickHouse read plane, an LLM orchestrates and explains, and three human confirm-gates govern every write — targeting hierarchy-to-finalized-plan under 1 hour (from days), ≥ 20 configurations explored per plan (from exactly 1), and run failures below 2% (from a measured 8.5%) with 100% reproducible shipped clusterings (from 0%).**

Backing. *Why:* clustering is the foundation of every assortment strategy plan yet the least assisted step — the system already computes attribute significance, then makes the human choose from raw score lists; users try **1 configuration per plan** in a search space of 12 algorithms $\times$ k 3–10 $\times$ any attribute subset; **8.5% of runs fail (37/437 on the kik tenant, >80% on input mistakes)** and nothing is reproducible (algorithm/params/seed never persisted) **[FRD-measured]**. *How:* invert the workflow — the user states **intent** (hierarchy + reference period, the only two non-derivable inputs); the agent handles scoping (active stores, sister-store substitution, sample-size guards), attribute selection (significance + coverage + redundancy gates $\rightarrow$ ranked feature sets), and batch exploration (20–100 pruned configs against an isolated scratch plan, results streaming as they score); the human confirms at **3 gates** and injects business knowledge (pins/moves). Approved results **write back into the existing product tables unchanged** — strategy/budget/wedge/line consume them with zero product changes. *Key design position:* the LLM never computes — **deterministic tools compute, the LLM orchestrates and explains**; same inputs + same scorer version $\Rightarrow$ identical ranking. *Tradeoff:* an agentic layer beside (not replacing) the manual wizard doubles surfaces, but de-risks adoption — both modes converge on the same config document, gates and write-back (the "convergence rule"). *Impact targets (6-mo):* <1 h to finalized plan, ≥ 20 configs, <2% failures, $\geq 60\%$ top-3 acceptance **[FRD-target]**. *Evidence:* Agentic Cluster FRD v1.8 §§0–2, 5, 6. *Honesty:* baselines are measured from live tenants; targets are committed design goals — say "measured baseline / design target."

C2 — The ClickHouse read plane (data-infrastructure story)

Cut agent data-probe latency from 1–20s+ (shared BigQuery slots, uncontrolled variance) to a p95 < 500 ms deterministic target by standing up a dedicated ClickHouse read plane fed by the existing BQ $\rightarrow$ ClickHouse ingestion lane — with agent access enforced read-only at the database-profile

level, nightly precompute converting $\geq 80\%$ of sessions from cold to warm, and cube-to-BigQuery reconciliation within 0.1%.

Backing. *Why:* an interactive copilot dies on 1–20s data probes; BigQuery's shared-reservation variance is the killer, and ad-hoc agent load on the shared slots hurts everyone else. *How:* reuse the org's proven BQ→ClickHouse ingestion pipeline (built for ItemSmart) to feed a **dedicated instance**; precompute significance + top candidates nightly for hierarchies with active plans; quote data-as-of in every recommendation; per-partition reconciliation guards derived-data rot. *Decision path:* this is the gated forward option from my reconciled DB verdict (L1) actually exercised — ClickHouse only for the read/pivot plane, BigQuery stays truth, Postgres stays the editable SoR; the FRD's non-goals state it explicitly ("NOT moving transactional data to ClickHouse"). *Tradeoff:* a derived copy that can rot (the org base rate is real) — mitigated by owned ingestion + freshness sentinel; and accepted physics on the ML itself (~9 s/config median model fit — we parallelize around it, we don't pretend to speed it up). *Impact:* P0 exit gate is p95 < 500 ms and 0.1% reconciliation [**FRD-target**]; compute baselines measured over 370 live runs (median job ≈ 20 s, max 463 s) [**FRD-measured**]. *Evidence:* FRD §§0, 3, 6–8; US-09/US-10.

C3 — Deterministic grounding (the anti-hallucination story)

Made chat-native intent safe by grounding every free-text mention deterministically — department names backtracked to full hierarchy paths via catalog search (never LLM guesses), season labels resolved against the tenant's fiscal calendar, ambiguity surfacing a clarifying question — with a confirm-required grounding card, so one prompt like "cluster Pet, Electrical and Cookware for Summer 2025" safely fans out to N parallel pipelines.

Backing. *Why:* the #1 way agent products lose trust is silently acting on a misparsed scope; hierarchy labels are ambiguous across levels and seasons differ per tenant and hemisphere. *How:* mentions → deterministic catalog backtracking to full paths (multiple departments → multiple filters); "Summer 2025" → fiscal-calendar + season-registry resolution with resolved dates displayed; the **grounding card** (scope, store cohort, sister-store substitutions, proposed attributes, each with a reason) must be confirmed before anything executes; batch intents get one disambiguation — *together or separately?* — then run as parallel pipelines bounded by a tenant concurrency budget. *Tradeoff:* an extra confirmation step per session vs. silent wrong-scope compute — cheap, and it doubles as the evidence surface. *Impact:* eliminates the input-error class behind >80% of today's run failures [**FRD-measured**]; acceptance criteria US-12/US-15. *Evidence:* FRD §2.1, US-12, US-15. *Interview gold:* "the LLM arranges the conversation; a catalog search decides what the words mean."

C4 — Reproducibility, pins & governed autonomy (the trust story)

Turned clustering from unreproducible one-shots into auditable, re-runnable artifacts — every shipped clustering persists a content-addressed config document (algorithm, k, params, seed,

features, windows, pins, data watermark, scorer version), planner store-moves become durable "pin" constraints honored by every future re-run (0% → 100% survival), and autonomy is a three-level delegation ladder (copilot → autopilot-with-approval → standing drift monitor) where no path ever auto-finalizes.

Backing. *Why:* today the winning configuration is thrown away (0% reproducible), and a planner's manual store swap is silently erased by the next re-run — human knowledge has no durable home **[FRD-measured]**. *How:* the **config document** is the unit of record — persisted, content-addressed, re-runnable ("re-run reproduces membership on the same watermark or explains drift"); **pins** persist as constraints with required reason codes and survive every refresh; delegation levels: L1 copilot proposes/executes on confirmation → L2 autopilot runs one intent end-to-end for a single review → L3 nightly drift monitor raises re-clustering proposals — and **system-originated work never self-finalizes** (surfaces as *pending intents* unless the tenant grants recorded standing pre-approval). A flow audit of 19 workflows produced a **closed-loop terminal-states contract** (every flow defines success / rejection / failure / expiry — nothing accumulates silently). Deviations from recommendations are captured with reasons and feed three honest learning loops: deterministic memory → human-approved calibration → labeled corpus for a future ranker (no silent self-tuning). *Tradeoff:* gates add friction vs. pure autopilot — chosen deliberately; `is_final` is a human signature. *Evidence:* FRD §§2.2–2.5, US-05/08/16/17, OD-1–OD-6. *Honesty:* this is design-you-own + spec'd acceptance criteria; implementation is phased P0–P3.

C5 — The strategy doorway (cross-module product thinking)

Closed the product's most-consumed dead-end — strategy plans must bind to a cluster plan at creation with zero help choosing (live audit: 1 in 5 finalized cluster plans had zero stores attached; briscoes' inventory 100% >17 months stale) — by shipping a deterministic ≤2s FitScore ranking with hard eligibility gates, per-candidate explain cards ("explains X% of store-revenue variance"), and an inline spawn path that runs the full recommendation pipeline inside strategy-create, backtested against 467 historical bindings.

Backing. *Why:* every strategy plan binds to exactly one cluster plan; the audit even found strategies pointing at cluster plans that no longer exist — a silent-breakage class. *How:* hard eligibility gate first (finalized ∧ not deleted ∧ non-empty membership ∧ recoverable scope — never score the unusable); **FitScore** ranks in ≤2 s (cheap terms for all candidates, the variance-explained term only for the top-5, computed on the last comparable season, penalized for cluster count, weighted by store coverage); bands (Green/Yellow/Red) **calibrated by backtesting against 363 kik + 104 briscoes historical bindings**; always a top-3 shortlist (padded with a spawn card — never an empty dropdown); spawn writes at the pre-finalize step and promotes only on human confirmation, or parks as a pending intent under tenant governance; every spawned plan records `created_via` + full scope keys so future doorways match it; and the create API now **validates bindings are live and finalized** — eliminating the dangling-reference class. *Impact target:* ≥40% of new strategy plans created with a suggested cluster plan **[FRD-target]**. *Evidence:* FRD §2.2, US-07. *Interview gold:*

the eligibility-gate stat — "1 in 5 finalized plans would silently break every downstream step; we made them unrankable."

2.3 Flagship — Data / database architecture

D1 — Three-tier data spine

Designed the platform's three-tier data spine — BigQuery (source of truth + batch cube builds) → Postgres (editable, ACID system-of-record) → ClickHouse/DuckDB (OLAP pivot & hindsight serving) — deliberately separating transactional grid edits from scan-heavy analytics instead of forcing one engine to do both.

Backing. *Why:* the product is "two shapes in one" — **OLTP-with-pivot** (an editable pivoted grid needing ACID + read-after-write) and **OLAP** (scan-and-aggregate hindsight). *No single engine is best at both.* *How:* BigQuery holds raw facts + builds the pre-aggregated planning cube via scheduled ELT; Postgres owns the editable plan (ACID, stored procs); DuckDB hydrates a plan slice for in-process pivots; ClickHouse is the gated read/pivot serving layer at scale. *Tradeoff:* more moving parts + a sync path vs. one-DB simplicity — justified because "everything in one OLAP DB" breaks transactional writes and per-edit cost, and "everything in BigQuery" hits DML quota/latency walls for interactive edits. *Decision path:* scored 4 engines across capabilities (writes, pivot latency, huge scans, cost) — Postgres wins writes, ClickHouse/DuckDB win pivots/scans — then assigned each its best job. *Impact:* each workload runs on the right engine; BigQuery stays truth in every scenario. *Evidence:* [docs/05](#) §7 scorecard; [db_architecture_poc/](#).

D2 — Insert-only / versioned ClickHouse (the key insight)

Unlocked ClickHouse as a planning store by designing an insert-only/versioned write model — **ReplacingMergeTree(version) + argMax(version)** reads, soft-delete, and atomic **REPLACE PARTITION** for bulk re-seeds — that holds the async mutation queue at zero, sidestepping ClickHouse's ~1,000-mutation ceiling while natively powering overrides, version-diff and undo/redo.

Backing. *Why:* ClickHouse is superb at OLAP pivots but its **ALTER UPDATE/DELETE mutations are async and degrade past ~500 / stop past ~1,000** — and a planning grid is edit-heavy, which is exactly why the *legacy* audit said "no ClickHouse." *Insight:* that objection is a property of the *legacy in-place-UPDATE* code, **not** of planning itself. The new grid is built around versions/variance/undo, so an edit can be an **INSERT of a new version** instead of an UPDATE. *How:* every cell edit inserts a row with **version = epoch-ms**; reads take **argMax(value, version)** where **is_deleted=0**; deletes insert a tombstone; bulk LY/LLY re-seeds use atomic **ALTER TABLE ... REPLACE PARTITION ... FROM staging**. A guard tracks **mutations_used** and asserts it stays **0**.

Tradeoff: history accumulates until background merges collapse duplicates (more storage, `FINAL / argMax` on read) — acceptable and it's exactly what powers undo/version-diff for free. *Impact*: converts the workload "from what ClickHouse is worst at to what it's fine at"; `mutations_used = 0` at 1x and 10x with correct read-after-write **[DESIGN/verified property]**. *Evidence*: `db_architecture_poc/domain/ch_planning_schema.sql`, `connectors/clickhouse.py`, `results/team_poc_results.json`, `docs/06 / 07`. *Honesty*: the *latency* ms in the PoC are mock; the *zero-mutation* property is the real, load-bearing result.

D3 — Tenant-unified schema at the derived grain

Standardized a tenant-unified schema at the derived (cube) grain — one common `plan_cube` partitioned by tenant (Postgres `LIST(tenant)`; ClickHouse `(tenant, l1, fiscal_week)`) — so one codebase, query set and stored procedures serve every client with no per-tenant tables.

Backing. *Why*: the legacy product forks a separate DB (and schema drift) per tenant, multiplying maintenance. *How*: define the cube grain once (`cluster × class × fiscal_week`), key every table by `tenant`, create one partition per tenant on onboarding; the tenant-parameterized cube-build SQL is identical across clients. *Tradeoff*: per-tenant *physical* isolation (separate Postgres DB + BigQuery project) is retained for security/blast-radius; "unified" is a **schema/codebase standard at the derived level**, not one shared physical instance. *Impact*: same queries/procedures everywhere → far less per-client code; onboarding a tenant = add a partition. *Evidence*: `domain/planning_schema.sql`, `domain/ch_planning_schema.sql`, `docs/06` R1.

D4 — Cube + DuckDB pivot accelerator

Built a pre-aggregated planning cube plus an in-process DuckDB pivot accelerator so cross-tab pivots read a small columnar cube slice instead of raw fact tables, keeping pivots interactive as the cube scaled 1x→100x in the benchmark.

Backing. *Why*: "pivot in the blink of an eye" over raw granular tables means re-scanning huge facts on every drag. *How*: materialize the cube in BigQuery, hydrate the plan's slice into an embedded DuckDB (Arrow), run native `PIVOT ... USING sum() GROUP BY`; commit deltas back to Postgres. *Tradeoff*: the cube must be rebuilt/refreshed (staleness vs. speed) — acceptable for a planning cadence. *Impact*: pivot cost tracks the *cube slice* size, not the fact table. *Evidence*: `services/{cube_builder,pivot_service}.py`, `connectors/duckdb_engine.py`, `results/benchmark_results.json`. *Honesty*: the ms are mock/indicative — frame as "kept pivots interactive as the cube scaled."

2.4 Cost / performance (mostly REAL audit numbers)

P1 — BigQuery cost program

Cut recurring BigQuery scan cost by up to ~280× (2.9 TiB → 3.3 GiB per query) by routing hierarchy reads to a clustered copy (~3×), materializing a store×category×week rollup (~280×; ~\$8 one-time + ~\$0.008/day), and enforcing partition pruning — after catching a single **SELECT *** query billing 38.8 TiB (\$327) to return 2 rows.

Backing. *Why:* BigQuery bills by **bytes scanned**; the hot recurring read rescanned the raw ~2.9 TiB fact. *How:* three levers — (1) read the clustered `_assort` copy, (2) build a narrow store×cat×week rollup, (3) require a date/partition predicate + kill **SELECT ***. *Tradeoff:* a rollup adds a build job + refresh; trivially paid back (~\$8 once). *Impact:* ~3× → ~74× (with category pruning) → ~280×/~880× (rollup) fewer bytes **[REAL, live audit]**. *Evidence:* `benchmark/cost_model.py`, `docs/07` F2–F5. *Honesty:* these are **real** measured scan sizes/costs — defend them.

P2 — BigQuery cost guard

Shipped a dependency-free "cost guard" in the query path that rejects **SELECT *** and un-dated scans and auto-routes raw-fact reads to the clustered copy — codifying the savings as a guardrail so future queries can't silently regress.

Backing. *Why:* one-off fixes rot; the leak recurs. *How:* a small linter around query execution (`bq_cost_guard` / `connectors/bigquery.py`) with knobs `FORBID_SELECT_STAR` / `REQUIRE_DATE_FILTER` / `CLUSTERED_FACT`. *Tradeoff:* a few legitimate queries need annotation to pass — worth it. *Impact:* makes the ~3× routing automatic and the biggest leak impossible. *Evidence:* `connectors/bigquery.py`, `services/bq_cost_guard.py`. *Honesty:* real mechanism; framed as guardrail.

P3 — Postgres write correctness & perf

Hardened the planning writes — wrapped multi-table saves in a single ACID transaction (eliminating ~88% orphaned optimizer rows), replaced N+1 per-row `jsonb_set` update loops with a set-based **UPDATE ... FROM json_to_recordset**, and added the missing cascade-delete FK indexes.

Backing. *Why:* non-atomic multi-statement saves left orphans; per-row loops on a 138-column partitioned table are the top write hotspot; un-indexed cascade FKs seq-scan on every plan delete. *How:* one transaction/array per save; one set-based statement per plan; three one-line `CREATE INDEX`. *Tradeoff:* minimal — pure win. *Impact:* correctness (no orphans) + far fewer statements per save. *Evidence:* the performance-findings doc (§2, §3.2); `docs/07` F12/F14. *Honesty:* orphan % is a **real** audit fact; the write rewrite is a recommended fix you designed.

2.5 Leadership / breadth

L1 — Reconciled DB verdict (decision under ambiguity)

Authored the reconciled database verdict across three conflicting analyses (a live "no ClickHouse" audit, a team ClickHouse push, and my own recommendation), converting disagreement into an explicit 3-gate decision — in-stack BigQuery fixes → Postgres discipline → a versioned-write PoC — that de-risked a costly migration before any infra spend.

Backing. *Why:* the org was split; a premature ClickHouse migration (or a premature "no") was expensive either way. *How:* separated *facts* (accepted from the live audit) from *conclusions*; showed the anti-CH argument was about legacy code, not the new product; set measurable gates. *Impact:* a defensible, staged path everyone could agree to. *Evidence:* docs/06 , docs/07 . This is your best behavioral/leadership story (see §7).

L2 — Reverse-engineered a 4-service agentic platform

Reverse-engineered a four-service agentic platform (React frontend, Rust/Axum data API, two Python/LangGraph LLM services) and delivered a fit-assessment + integration plan (~1–1.5 eng-weeks to add the decomposition node), plus a catalog-verified data model (5,385 columns, 46 FKs) via \$0 read-only metadata.

Backing. *Why:* to plan the agentic rebuild you must first understand the existing system truthfully. *How:* code-level read of CortexEye (genai + cortexeye-ai LangGraph brains, Rust backend, React FE), then a metadata-only verification of the DB (Postgres pg_catalog + BigQuery get_table). *Impact:* corrected long-standing "inferred" assumptions and produced an actionable plan. *Evidence:* docs/01 – 03 ; the data deep-dive §1.8/§2.10. *Honesty:* verification numbers are **real**; keep the "reverse-engineered/authored" framing.

3. Product architecture (HLD) — how the agentic planner is built

3.1 The planning workspace (front door)

Four surfaces sit over the staged pipeline:

- **Signals** — the agent worklist: issues/opportunities/proposals, each Accept / Skip / Edit / Dismiss with a **reason code**.
- **Pipeline** — the staged plan workflow with **cascade warnings** when an upstream edit invalidates downstream work.
- **Log** — the immutable **audit trail** (who/what/when) for every agent action + human decision.
- **Ask** — the natural-language assistant; the UX front door to the multi-agent decomposition backend.

Plus a **Today** triage bucket (Attention / Resolved / Dismissed) and **overnight** autonomous-action summaries.

3.2 Cascade-aware staged planning

```
Hindsight → Strategy → Clustering → Line plan / Architecture → Buy plan → Review & approve
(last season) (targets) (store groups) (option counts/depth) (qty/receipts/flow)
| editing an upstream stage raises a cascade warning: downstream stages recompute/invalid
```

Each stage is agent-assisted; `getDownstreamStages()` + `showCascadeWarning()` enforce that changing, say, clustering flags that the buy plan must be recomputed.

3.3 Agent orchestration (the backend shape)

Concrete pattern: **Router** → **Orchestrator (supervisor)** → **specialist agents**, executing a **planner-executor DAG**. In the product this runs on **LangGraph** (Planner → Executor → Solver nodes) with a **FastMCP (Model Context Protocol)** tool layer so agents discover/execute tools safely; the offline `agentic_assort_multiagent` PoC re-implements the same shape deterministically for testability (see §8.1).

```
NL query
|
▼
[Semantic cache]—hit→ cached answer
| miss
▼
ROUTER (LLM intent classify) — single → one specialist agent → answer
| orchestrate
▼
DECOMPOSER → Plan{sub_queries[], path[][]} → plan_schema.validate()
|                                     | invalid
|                                     ▼
|                                     self-repair (re-prompt once)
|
▼
ORCHESTRATOR executes path stage-by-stage:
  stage 0: [agentA || agentB]      ← parallel within a stage (ThreadPoolExecutor)
  stage 1: [agentC(<A0>)]          ← <A_n> placeholder rephrased with prior answers
  ...
```


▼
_synthesize() → final answer (+ status: ok | partial | rejected | generic)

Specialist fleet (5 agents), each **validate→fetch→analyze→narrate** :

Agent	Store	Job
AssortmentData	BigQuery fact_sales	KPI snapshots, top/bottom-N
Clustering	Postgres store_cluster	cluster membership / tier / profile
Hindsight	BigQuery causal_drivers	rank KPI drivers by contribution (causal DAG "why")
LinePlan	Postgres line_plan	option counts / depth / phasing, Δ vs LY
Recommendation	BigQuery ros_recommendations	ROS-based option-count uplift + gap flags

Resilience: `BaseAgent.answer()` never raises → `AgentResult(status="error")` → orchestrator still synthesizes a **partial** answer; transient `fetch()` wrapped in exponential-backoff retries.

3.4 The pivot planning grid (edit surface)

A React grid (`planning-grid-3.jsx`) with: drag **dimension tiles** into Row/Column/Filter axes (`PivotPanel` / `AxisZone` / `DimTile`); **Override** / **Variance** / **Diff** cells (`0vCell` / `VarCell` / `DiffCell`); **BulkEdit** (apply %, set, spread); **reason capture**; **downstream impact** panel; **version drawer** (compare/restore); and a custom `useUndoRedo` history. This grid is *why* the persistence layer must support versions/variance/diff — which is exactly what the insert-only ClickHouse model (below) provides natively.

3.5 The first shipping module — the Cluster Recommendation Copilot

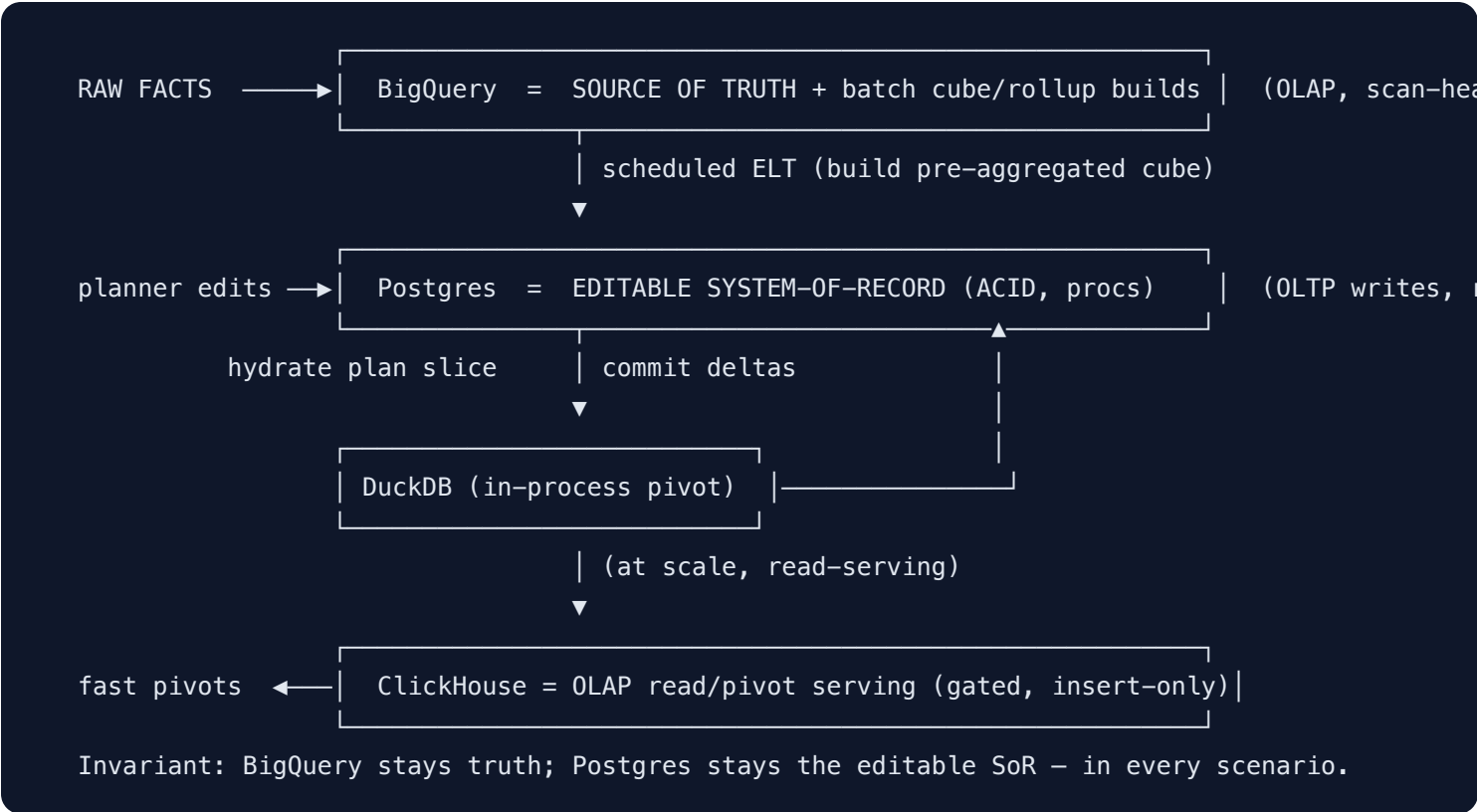
The agentic pattern above is now being instantiated for real in the **clustering module** (FRD v1.8): one dedicated agent, two entry surfaces (manual wizard with inline recommendations + chat-native), **deterministic tools on a dedicated ClickHouse read plane** (p95 <500 ms target vs 1–20 s on shared BigQuery slots), an LLM that orchestrates and explains but never computes, **three confirm gates**, and write-back into the existing product tables unchanged. Delegation ladder: **L1 copilot** → **L2 autopilot-with-approval** → **L3 standing drift monitor** — nothing auto-finalizes. Full what/why/how, measured baselines, targets, rollout phases and module-specific Q&A: §9.

3.6 The wider CortexEye agentic platform (context you can speak to)

The agentic backend descends from CortexEye — a **4-service** system: - **Frontend** — React + Redux, **Socket.IO** streaming, AG Grid, Highcharts. - **Data API** — Rust (**Axum / Tokio / SQLx**), **gcp-bigquery-client** — the deterministic data layer (no LLM). - **genai** — Python + **LangGraph** + FastAPI: intent classification (**causal > data > insights**), entity extraction (parallel via **asyncio.gather**), **text-to-SQL** (LlamaIndex RAG + GPT-4o) on BigQuery, **confidence scoring** (rules + LLM logprobs). - **cortexeye-ai** (Deep Research) — Python + LangGraph: the strong **sub-query decomposer** (**{question[], path[][]}** + **<A_n>** dependency rephrasing + reflection loop), hybrid **BM25 + dense (pgvector) retrieval + Cohere rerank**, semantic cache. Cross-cutting: **Redis** (chat history, semantic cache, abort flags across Cloud Run replicas), **Postgres via PgBouncer** (**aiopg**), **GCS** (audio/images), progress streamed over **WSS**.

4. Database architecture (HLD + LLD)

4.1 HLD — the three-tier spine



Why split: the product is **OLTP-with-pivot** (editable grid, ACID, read-after-write) *and* **OLAP** (hindsight scans). No single engine is best at both, so each workload runs on the engine that's best at it. "Everything in one OLAP DB" breaks transactional writes + per-edit cost; "everything in BigQuery" hits DML quota/latency walls for interactive edits.

4.2 The tenant-unified derived schema

One common schema keyed by `tenant`, at the **derived (cube) grain** = `cluster × class × fiscal_week` — **not** per-tenant tables.

- **Postgres** (`planning_schema.sql`): `plan_cube PARTITION BY LIST (tenant)`, PK (`tenant, season, plan_id, cluster, class, fiscal_week`); onboarding a client = `CREATE TABLE plan_cube_kik PARTITION OF plan_cube FOR VALUES IN ('kik')`. Same-shaped `plan_cell_edits` audit table.
- **ClickHouse** (`ch_planning_schema.sql`): `tenant` leads both `PARTITION BY (tenant, l1, fiscal_week)` and `ORDER BY (tenant, season, plan_id, l4, store_code, metric)`.
- The **cube-build SQL is tenant-parameterized and identical** across clients → one codebase/query set/procedures serves all tenants. Physical isolation (separate Postgres DB + BigQuery project per tenant) is retained; "unified" is a **schema/codebase standard at the derived level**, not a shared physical instance.

4.3 LLD — ClickHouse insert-only / versioned model

```
-- main planning cube: INSERT-ONLY VERSIONED (no ALTER UPDATE/DELETE)
CREATE TABLE plan_cube (
  tenant String, season String, plan_id String,
  l1 String, l4 String, store_code String,
  fiscal_week UInt16,
  metric String, value Float64,           -- 80-90 metrics modelled long (metric,value)
  version UInt64,                         -- monotonic epoch-ms; newest wins
  is_deleted UInt8 DEFAULT 0              -- soft delete (no hard DELETE)
) ENGINE = ReplacingMergeTree(version)
PARTITION BY (tenant, l1, fiscal_week)    -- week/L1 anchor avoids hot-partition churn
ORDER BY (tenant, season, plan_id, l4, store_code, metric);

-- EDIT = insert a new version (never UPDATE)
INSERT INTO plan_cube VALUES (... , metric, new_value, toUnixTimestamp64Milli(now64()), 0);

-- READ latest = argMax over version (or FINAL on small partitions)
SELECT l4, store_code, argMax(value, version) AS value
FROM plan_cube WHERE tenant=? AND plan_id=? AND is_deleted=0
GROUP BY l4, store_code;

-- SOFT DELETE = insert a tombstone version
```

```

INSERT INTO plan_cube VALUES (... , metric, 0.0, <ver>, 1);

-- BULK re-seed (LY/LLY across a slice) = atomic partition swap, NOT a mutation
ALTER TABLE plan_cube REPLACE PARTITION (tenant,l1,week) FROM plan_cube_staging;

-- incremental brand/L1 rollups without rescans
CREATE TABLE plan_cube_l1_rollup ( ..., value_state AggregateFunction(sum, Float64) )
ENGINE = AggregatingMergeTree ...;

```

Why this is the crux: ClickHouse `ALTER UPDATE/DELETE` mutations are async — they degrade past ~500 and stop accepting past ~1,000. A planning grid is edit-heavy, which is why the legacy audit said "no ClickHouse." The insight: **model an edit as an INSERT of a new version**, read the latest with `argMax(version)`, tombstone for deletes, partition-swap for bulk re-seeds. A guard asserts `mutations_used == 0`. This also gives **undo / version-diff for free** (older versions retained until merge; the product caches up to 10 undo steps in Redis before async commit, rendering diffs as dual-value cells). Production hardens it with `ReplicatedReplacingMergeTree` + **sharding** (ClickHouse Keeper, or physical **L1-table sharding to kill hotspots**), `AggregatingMergeTree` rollups, `VersionedCollapsingMergeTree` for out-of-order CDC, and **Projections**; partition `PARTITION BY toYYYYMM(week_date)`, `ORDER BY (measure, hierarchy...)` for sparse-index granule skipping. **PoC results (requirements notebook):** `mutations_used = 0` at every scale, and **sub-second pivots over 250M synthetic rows — 8–130 ms filtered reads, ~160 ms writes, ~140–200x vs the legacy Postgres flat schema** (see §8.4–8.5).

4.4 Cube / rollup / pivot (fast reads)

- **cube_builder** — runs the tenant-parameterized `CUBE_BUILD_SQL` on BigQuery (aggregates fact → `cluster×class×week`), upserts to Postgres `plan_cube`.
- **rollup_builder** — CTAS a `store×category×week` rollup **from the clustered fact** (`PARTITION BY RANGE_BUCKET(fiscal_week) CLUSTER BY store_code, l3_name`): reads ~3.3 GiB instead of ~2.9 TiB (~280–880x cheaper; ~\$8 once, ~\$0.008/day).
- **pivot_service** — loads the plan's cube slice into DuckDB (Arrow) and runs native `PIVOT ... USING sum() GROUP BY`; pivots track the *slice* size, not the fact.

4.5 The reconciled verdict + decision gates

The decision was **not** "adopt ClickHouse now" — it was a staged gate: 1. **In-stack BigQuery fixes first** (certain, no new infra): clustered `_assort` (~3x), `store×cat×week` rollup (~280x), forbid `SELECT *` + require date filter (kills the 38.8 TiB / \$327 leak), fix cross-tenant hardcodes. 2. **Postgres discipline**: transaction-wrap multi-table ops, set-based `UPDATE ... FROM`, repartition `size_split_master` (5,503 LIST partitions for 298K rows), purge soft-delete debris, normalize the EAV spine. 3. **Versioned-write + pivot PoC at 10x**: measure Postgres+DuckDB vs ClickHouse on the *same* insert-only cube; **only then** decide if ClickHouse earns a place.

Two end-states: (1) **ClickHouse-unified** if the insert-only model meets the pivot+update SLA at 100–200 users; (2) **split SoR** (BigQuery truth + Postgres strict-edit SoR + DuckDB pivot; CH read-only) if strict in-transaction per-cell read-after-write is required. *Anti-pattern explicitly rejected: "everything in one OLAP DB."*

4.6 Ingestion (be precise in the room)

Ingestion into the serving/planning stores is **batch ELT, not streaming**: BigQuery cube/rollup builds → upsert/COPY into Postgres; ClickHouse fed by HTTP `INSERT ... FORMAT JSONEachRow` + `REPLACE PARTITION` from staging. **No Kafka/Airflow/Spark/Flink in the Agentic-Assort DB path.** (Elsewhere in the org: the *data-fabric* platform uses an **aiokafka** consumer for async embeddings and **Cloud Tasks + Cloud Scheduler** for orchestration; legacy AssortSmart uses **Pub/Sub** for MDM sync — mention these as adjacent, not as this product's pipeline.)

5. Technology deep dives (funda · architecture · why it solves the problem · gotchas)

Each entry is what you should be able to say if an interviewer points at a word on your résumé.

5.1 ClickHouse

- **Fundamentals.** Open-source **columnar OLAP** DB. Column-oriented storage + vectorized execution → aggregations scan only needed columns, compress well, and run SIMD-fast. Data lives in immutable **parts** (sorted by the table's `ORDER BY`); a background process **merges** parts (the "MergeTree" family).
- **Architecture (engines).** `MergeTree` = base (sorted parts, partitioning, sparse primary index).
`ReplacingMergeTree(version)` = on merge, keep the **highest version** per sort key → dedup by newest. `AggregatingMergeTree` = store partial aggregate states (`AggregateFunction`) merged incrementally. `Distributed / Replicated*` = sharding/replication (we defer to ClickHouse Cloud).
Partitioning (`PARTITION BY`) prunes at query time; `ORDER BY` is the primary/sort key that clustering + the sparse index rely on. **Projections & materialized views** precompute alternative sort orders/rollups.
- **How it solves our problem.** Pivot + hindsight are scan-and-aggregate → columnar is ideal. Its weakness — **async mutations** (`ALTER UPDATE/DELETE` degrade >500, stop >1000) — is avoided by our **insert-only/versioned** model: edit = insert a new `version`, read = `argMax(version)`, bulk = `REPLACE PARTITION`. So we use the engine only where it's strong and never touch the mutation queue.
- **Deep-dive/gotchas.** `argMax` (or `FINAL`) is needed on read because dedup only happens *eventually* on merge; keep partitions from getting too many parts (hot-partition churn) — we partition by (`tenant`, `l1`, `fiscal_week`); `SELECT FINAL` is convenient but costs a merge-on-read; `AggregatingMergeTree`

requires `-State` / `-Merge` function discipline. It is **not** for per-row transactional read-after-write with locks — that's Postgres's job.

5.2 BigQuery

- **Fundamentals.** Serverless columnar warehouse; storage (Capacitor columnar) separate from compute (**slots**). You're billed **on-demand by bytes scanned** (or by slot reservations). No secondary indexes.
- **Architecture. Partitioning** (usually by date) + **clustering** (sort within partitions by up to 4 cols) are the only "indexes." Query cost = columns read × partitions touched. DML (`UPDATE/MERGE`) is quota-limited and rewrite-oriented → not for interactive edits.
- **How it solves our problem.** Perfect **source of truth** + batch cube/rollup builder: cheap, scalable scans; scheduled ELT. We keep planner edits *out* of BigQuery.
- **Deep-dive/gotchas.** The three levers that gave ~3x/~74x/~280x: (1) read the **clustered** copy not the raw fact, (2) always pass a **partition/date predicate** (a bypass floor like `date > '1999-01-01'` defeats pruning), (3) **never** `SELECT *` (one `SELECT * LIMIT 2` billed 38.8 TiB / \$327). Set `require_partition_filter=TRUE` so un-pruned scans fail loudly.

5.3 PostgreSQL

- **Fundamentals.** Row-store RDBMS, **MVCC** (each write creates a new tuple version; `VACUUM` reclaims dead tuples), full ACID, rich indexing (btree, GIN, partial, expression), declarative **partitioning**, `jsonb`.
- **Architecture in our stack.** The **editable system-of-record**: plan masters + the editable grid as row tables with stored procs; LIST-partitioned by tenant / `final_level`; **PgBouncer** pools connections (`aiopg` async). GIN index on `jsonb` attribute bags; composite btrees for hot filters.
- **How it solves our problem.** ACID + read-after-write + locking = exactly what a per-cell planning edit needs (the thing OLAP engines are bad at).
- **Deep-dive/gotchas.** MVCC + del+reinsert = **dead-tuple bloat** (needs `VACUUM`); a `jsonb ->>` filter can't use a `jsonb_path_ops` GIN (rewrite as `@>`); **always include the partition key** or you scan every partition; cascade-delete FK columns must be indexed or each parent delete seq-scans children; prefer **set-based** `UPDATE ... FROM json_to_recordset` over N+1 per-row loops; over-partitioning (5,503 partitions for 298K rows) hurts planning time.

5.4 DuckDB

- **Fundamentals.** Embedded ("SQLite for analytics") **columnar, vectorized** OLAP engine; runs in-process, reads Arrow/Parquet zero-copy.
- **How it solves our problem.** Per-plan/per-session **pivot accelerator**: hydrate the plan's small cube slice into DuckDB, run native `PIVOT ... USING sum() GROUP BY`, commit deltas to Postgres — fast cross-tabs without a server round-trip or rescanning facts.

- **Deep-dive/gotchas.** Great single-node/embedded; not a multi-user server (that's why it's session-scoped and Postgres/ClickHouse own durability + concurrency).

5.5 Agent orchestration (router / planner-executor / supervisor) & LangGraph

- **Fundamentals.** **LangGraph** models an agent app as a **state graph** (nodes = steps/agents, edges = control flow, a shared typed **state** object) — good for cyclic, multi-step, human-in-the-loop flows vs. a linear chain. The classic patterns: **router** (classify then dispatch), **planner-executor** (plan a DAG, then execute), **supervisor/orchestrator** (a controller delegates to workers and synthesizes).
- **Architecture (ours).** Router → Decomposer (emits a validated DAG) → Orchestrator executes stages (parallel within stage) → synthesize. CortexEye's production brains are LangGraph; our multi-agent PoC is a **custom** Python router/orchestrator (stdlib `concurrent.futures`) that mirrors that shape deterministically for testability.
- **How it solves our problem.** A planning question needs several specialized data pulls with dependencies; a validated DAG + supervisor gives parallelism, dependency passing (`<A_n>`), partial-failure tolerance, and an auditable trace.
- **Deep-dive/gotchas.** The **#1 failure is a malformed plan** → a hard structural validator + one self-repair pass is the fix; guard against forward references; keep stages ≤3 agents wide, ≤8 sub-questions; make each agent's `answer()` non-throwing so one failure yields a *partial*, not a crash.

5.6 LLM plumbing (text-to-SQL, decomposition, confidence, RAG, cache)

- **Text-to-SQL.** LLM emits SQL from NL + schema (LlamaIndex RAG over schema in CortexEye). **Guardrails are the engineering:** SELECT-only, `LIMIT` caps, `NULLIF` denominators, no `SELECT *`, validate before execute. Solves "let business users ask in English" without runaway scans.
- **Query decomposition.** One strong prompt (18 worked examples) → `{sub_queries[], path[][]}` DAG + `<A_n>` dependency rephrasing + a reflection/repair loop. Solves multi-hop questions ("why did margin drop and which clusters?") that a single query can't.
- **Intent classification / confidence.** A deterministic rules prompt at `temperature=0` (chosen over a RAG/few-shot classifier for **latency, cost, determinism on novel queries**); confidence via **rules + LLM logprobs + intent-match + error detection**. Solves routing + "should we trust/show this answer."
- **RAG (hybrid).** `text-embedding-3-small` → **pgvector**; **BM25 (sparse) + dense** retrieval fused, then **Cohere rerank**. Hybrid beats either alone (lexical + semantic); rerank fixes ordering.
- **Semantic cache.** Embed the query; return a cached answer when cosine-sim ≥ threshold within TTL (pgvector `<=>` in prod). Solves repeat/near-duplicate questions → skip the whole pipeline. Gotcha: threshold too loose returns wrong-but-similar answers — tune + TTL-bound.

5.7 Rust / Axum (CortexEye data API)

- **Fundamentals.** Rust = memory-safe, no-GC, high-throughput; **Axum** (on **Tokio** async runtime, **SQLx** compile-checked queries) is the HTTP framework.
- **How it solves our problem.** A **deterministic, fast data API** layer (BigQuery/GCS access, TTS streaming) separate from the probabilistic Python LLM services — clean separation of "deterministic data" vs "AI reasoning," and Rust's throughput/safety for the hot data path.
- **Deep-dive.** Compile-time-checked SQL (SQLx) prevents a class of query bugs; async I/O (Tokio) for high concurrency; keeps LLM latency/cost out of the core data path.

5.8 Supporting infra

- **FastAPI + asyncio** — async Python services; `asyncio.gather` to run classification + entity-extraction (or independent reads) concurrently.
- **Redis** — chat history, **semantic cache**, and **abort flags** (cancel a running task across Cloud Run replicas). Fundamentals: in-memory key-value, sub-ms, TTL, pub/sub.
- **Socket.IO / WebSockets (WSS)** — stream agent progress/partial tokens to the UI (long LLM calls need incremental feedback).
- **Google Cloud Tasks + Scheduler** — async job offload + cron for ELT/cube builds (keeps heavy work off the request path).
- **Secret management** — Google Secret Manager, IA convention
`{client}_ms_ce_chatbot_{provider}_{env}` ; ADC auth; no keys in code.
- **Apache tech (what the product actually uses).** **Apache Kafka** (`aiokafka` / `confluent-kafka`) drives **async embedding jobs**, and **Apache Parquet** is the columnar storage/interchange format (§8.3). The **planning-store ingestion path** itself is **batch ELT** (BigQuery cube/rollup → Postgres/ClickHouse), not a streaming pipeline into the grid — so be precise: "Kafka for async embedding work + Parquet for storage; the grid's data lands via batch ELT." Kafka fundamentals to know: distributed append-only commit log, topics→partitions, consumer groups, at-least-once + offsets. If asked "where else would Kafka fit?": streaming POS/inventory events into the fact, or a CDC→ClickHouse path (ClickPipes / Kafka engine).

6. Interview Q&A (what an EM / architect will ask)

6.1 Architecture & databases

Q: Why three databases? Isn't that over-engineered? A: The product is two workloads in one — an editable, pivoted **planning grid** (OLTP: ACID, per-cell read-after-write, locks) and **hindsight analytics** (OLAP: scan-and-

aggregate). No single engine is best at both. BigQuery is the cheap, scalable source of truth; Postgres owns editable state with ACID; ClickHouse/DuckDB serve fast pivots. Forcing one engine means either broken transactional writes (OLAP DBs) or DML quota/latency walls (BigQuery). The complexity is a deliberate trade for correctness + cost + latency, and the invariant is simple: **BigQuery = truth, Postgres = editable SoR, always.**

Q: Why ClickHouse if it's bad at updates — and a planning grid is all updates? A: That objection is about *in-place* updates. ClickHouse mutations are async and stop accepting past ~1,000, so `ALTER UPDATE` per cell is a non-starter. But our grid is built around **versions/variance/undo** — so an edit is an **INSERT of a new version**, reads take `argMax(version)`, deletes are tombstones, bulk re-seeds are atomic `REPLACE PARTITION`. That keeps the mutation queue at **zero** and turns "what ClickHouse is worst at" into "what it's fine at," while giving undo/version-diff for free. We proved `mutations_used = 0` at 1x and 10x in the PoC.

Q: How does undo / version-diff actually work on an append-only store? A: Every edit writes a row with a monotonic `version` (epoch-ms). The latest state is `argMax(value, version)`. Because older versions are retained until background merges collapse them, **undo** = read the prior version; **version-diff** = compare two `version` snapshots. `ReplacingMergeTree(version)` dedups newest-wins on merge; `FINAL / argMax` gives correct reads before merge.

Q: What does "unify the schema at the derived level" mean? A: One common schema keyed by `tenant` at the **cube grain** (`cluster×class×fiscal_week`), not per-tenant tables. The cube-build SQL is tenant-parameterized and identical, so one codebase/query set/procedures serve all clients; onboarding a tenant = add a partition. Physical isolation (separate Postgres DB + BigQuery project per tenant) is kept for blast-radius; "unified" is a **schema/codebase standard**, not one shared instance.

Q: How did you get ~280x cheaper on BigQuery? A: BigQuery bills by bytes scanned. Three levers: route the hot hierarchy read to the **clustered _assort** copy (~3x), materialize a narrow **store×category×week rollup** (reads ~3.3 GiB vs ~2.9 TiB → ~280x; ~\$8 to build, ~\$0.008/day), and enforce **partition pruning + no SELECT ***. We also caught a `SELECT * LIMIT 2` billing 38.8 TiB / \$327 — fixed by a cost-guard that rejects `SELECT *` and un-dated scans.

Q: How do you keep the three stores in sync? A: BigQuery → batch ELT builds the cube → upsert/COPY to Postgres (SoR). Planner edits commit to Postgres; DuckDB pivots a hydrated slice and commits deltas back. ClickHouse is fed read-only from BigQuery cube/rollup builds via HTTP insert / partition-swap. It's **batch, not streaming** — planning has a cadence, not a millisecond SLA.

6.2 AI / agents

Q: Why multiple agents instead of one big prompt? A: Separation of concerns + reliability. Each agent owns a domain (hindsight, clustering, line plan, recommendations, assortment data), its own tools, prompt, and validation. A router decides single-agent vs orchestrate; a decomposer builds a DAG. This gives parallelism, targeted prompts (better accuracy), independent failure handling, and an auditable trace — impossible in one monolith prompt.

Q: LLM plans are unreliable — how do you stop bad plans from executing? A: A hard **structural validator** runs before execution: it rejects unknown domains/tools, duplicate ids, over-wide stages, and forward references in `<A_n>` placeholders. On failure it re-prompts the LLM once with the exact errors and re-validates (`repaired=True`). In an offline eval this lifted decomposition quality from 0.54→0.99. The validator — not the model — is what makes it production-safe.

Q: What happens when an agent fails? A: `BaseAgent.answer()` never raises; a failure returns `status="error"`, transient I/O is retried with exponential backoff, and the orchestrator still **synthesizes a partial answer** from the agents that succeeded (clearly labeled). Degrade, don't crash.

Q: How do agents pass data to each other? A: The DAG has stages; agents within a stage run in parallel (thread pool). When a later sub-query depends on an earlier answer, a rephraser substitutes the `<A1>` placeholder with the prior result before dispatch — explicit, inspectable dependency passing rather than hidden shared state.

Q: How do you keep autonomous agents safe? A: Confidence/impact **thresholds**: below threshold agents act and log; above it they raise a human-in-the-loop **Signal** with a reason code for Accept/Edit/Dismiss. Every action + decision hits an immutable **audit log**. That's what makes overnight autonomy acceptable to merchants.

Q: Is text-to-SQL safe? A: Only with guardrails: SELECT-only, `LIMIT` caps, `NULLIF` denominators, no `SELECT *`, validate before execute, and route through the BigQuery cost-guard. The LLM proposes; the guardrails dispose.

Q: How do you measure agent/answer quality? A: An offline eval harness scores routing, decomposition structure, dependency capture, and coverage against gold queries (structured 0.99 vs flat baseline 0.54). For confidence at serve time: rules + LLM logprobs + intent-match. (I'm explicit that the harness measures structure/correctness on mock outputs, not live model accuracy — production would add human-rated eval sets + online metrics.)

Q: You keep saying "agentic" — what's actually shipping? A: The clustering module — a cluster-recommendation copilot spec'd against live-audit baselines (1 config tried per plan, 8.5% run failures >80% of which are input errors, 0% reproducibility). The planner states intent — hierarchy + period — and a dedicated agent does scoping, attribute selection and a 20–100-config batch exploration on a dedicated ClickHouse read plane; three human gates; approved results write back into the existing tables unchanged. Targets: <1 h to a finalized plan, ≥20 configs, <2% failures, p95 <500 ms probes. Full module Q&A in §9.7 — including "why isn't this just AutoML," "how do you stop hallucinated recommendations," and "what did you deliberately not build."

6.3 Tradeoffs, scale, "what would you do differently"

Q: Biggest tradeoff you made? A: Accepting **more infra + a sync path** (three tiers) to get correctness + cost + pivot latency, instead of one-DB simplicity. And on ClickHouse: trading **storage + merge overhead** (versioned history) for zero mutations + free undo/diff.

Q: How does this scale to 20k stores? A: Cube pre-aggregation keeps pivots proportional to the *slice*, not the fact; ClickHouse partition/cluster + `argMax` reads are sub-second on ~900M rows (cited target); Postgres stays the editable SoR with proper partitioning. The gate is a **10x PoC** on the insert-only model before committing ClickHouse.

Q: What would you do differently / next? A: Add circuit-breakers + per-agent timeouts; a human-rated eval set for model accuracy (not just structure); wire the `AggregatingMergeTree` materialized views end-to-end; and evaluate a CDC/Kafka path if we need near-real-time facts. I'd also fix the legacy BigQuery leaks first (cheapest, certain win) before any ClickHouse spend — which is exactly the staged gate I recommended.

Q: How did you decide ClickHouse vs not, given the team disagreed? A: I separated **facts** (accepted from the live audit) from **conclusions**, showed the anti-ClickHouse argument was about *legacy in-place-update* code not the new product, and converted the disagreement into a **3-gate decision** (in-stack BQ fixes → Postgres discipline → versioned-write PoC). Nobody had to "win" — we agreed on measurements.

6.4 Behavioral (use these as STAR seeds)

- **Decision under ambiguity / conflict:** the reconciled DB verdict (three conflicting sources → one staged gate). (§2 L1)
- **Ownership / de-risking:** the BigQuery cost program + cost-guard (found a \$327-for-2-rows leak; codified the fix). (§2 P1/P2)
- **Depth / correctness:** catalog-verified 5,385 columns / 46 FKs via \$0 metadata, overturning "inferred" assumptions. (§2 L2/P4)
- **Invention:** the insert-only/versioned ClickHouse model that made an OLAP engine viable for an edit-heavy grid. (§2 D2)

7. Talking tracks, tailoring & honesty

7.1 Three pitch lengths

30 seconds: "I build the AI and data backbone of an agentic retail assortment planner — right now the store-clustering copilot: the planner states what to cluster and for when, a dedicated agent handles scoping, attribute selection and a 20-to-100-configuration exploration on a dedicated ClickHouse read plane, with evidence packs and three human approval gates. It's spec'd against measured baselines — 8.5% run failures, one config tried per plan, zero reproducibility — with targets of under an hour to a finalized plan and sub-500 ms agent probes. Under it sits a three-tier data architecture (BigQuery, Postgres, ClickHouse/DuckDB) I designed, and a BigQuery cost program that cut recurring scan cost up to ~280x."

2 minutes: add the *why* — clustering is the foundation of every strategy plan yet the least assisted step (the system computed attribute significance and then ignored it); the workflow inversion (human states intent, agent derives everything derivable, human confirms at gates and injects pins); the governance ladder (copilot → autopilot-with-approval → drift monitor; nothing auto-finalizes; content-addressed config documents make every shipped clustering reproducible); two workloads in one product (OLTP-with-pivot + OLAP) → three tiers; the ClickHouse insight (edit = insert-a-version → mutation queue stays at zero → undo/diff for free); the agent orchestration (router → decomposer → validated DAG → parallel specialists → synthesize, with a structural validator that fixed the #1 LLM failure); and the staged decision gate that de-risked the migration.

Deep dive: walk §9 (the copilot — measured problem → inversion → architecture positions → doorway), then §4 (DB LLD) and §3 (agent DAG), then the numbers in §2 (real vs indicative), then the tradeoffs in §6.3.

7.2 STAR stories (ready to tell)

- **S/T:** Team split on whether the new planner should adopt ClickHouse; a live audit said "no," a SME push said "yes." **A:** Separated facts from conclusions, showed the objection was about legacy in-place-update code, designed an insert-only/versioned model + a 3-gate decision. **R:** Agreed, measurable path; PoC proved zero mutations at 10x; no premature infra spend.
- **S/T:** Hindsight/pivot reads were slow and BigQuery spend was high. **A:** Audited scan patterns; routed to clustered copy, built a rollup, added a cost-guard. **R:** up to ~280x fewer bytes; caught a 38.8 TiB / \$327 leak; codified as a guardrail.
- **S/T:** LLM planners produced malformed multi-step plans. **A:** Added a structural DAG validator + one self-repair pass + `<A_n>` dependency passing. **R:** decomposition 0.54→0.99 in an offline eval; graceful partial answers on agent failure.
- **S/T:** Clustering — the foundation of every strategy plan — was the least assisted step: 1 config tried per plan, 8.5% of runs failing on avoidable input errors, nothing reproducible, planner swaps silently erased. **A:** Spec'd and started building the cluster-recommendation copilot: intent-only input, deterministic grounding, batch exploration on a dedicated ClickHouse read plane, evidence packs, three gates, pins as durable constraints, content-addressed config documents. **R:** committed targets — <1 h to a finalized plan, ≥20 configs, <2% failures, 100% reproducibility, p95 <500 ms probes — with write-back into the existing product unchanged.

7.3 Tailoring to the JD

- **AI/ML-platform role** → lead with C1 + C3 + A1–A4 + §5.5/5.6 (the shipping copilot, deterministic grounding, orchestration, decomposition, text-to-SQL, RAG, confidence).
- **Data/infra/DB role** → lead with C2 + D1–D4 + P1–P3 + §4 (dedicated ClickHouse read plane, three-tier spine, insert-only ClickHouse, cost program).
- **Backend/platform role** → lead with D1, A2, P3 + §5.7/5.8 (Rust API boundary, async, Redis, streaming, Cloud Tasks).

- **Staff/architect signal** → lead with C1 + C4 + L1 (a governed agentic system you own end-to-end, the reconciled verdict / decision gates) + the tradeoff answers in §6.3.
- **Agentic/AI-product role** → lead with C1–C5 wholesale + §9 (intent taxonomy, delegation levels, terminal-states contract, learning loops). Keep the résumé to the 5–8 strongest for that JD; the rest live here for the interview.

7.4 Honesty guardrail (repeat — this protects you)

Claim	Tier	Say it as
BigQuery ~3× / 74× / 280× / 880×; 38.8 TiB / \$327 leak	REAL (live audit)	"measured" / "cut"
Postgres 88% orphaned rows; 5,503 partitions / 298K rows	REAL (audit)	"found / fixed"
Catalog verified: 5,385 columns, 46 FKs	REAL (metadata)	"verified"
<code>mutations_used = 0</code> at 1×/10× (insert-only works)	REAL property (PoC)	"proved the architecture property"
Decomposition 0.54 → 0.99 (+0.45)	EVAL (offline, mock model)	"in an offline evaluation harness"
Pivot/seed/edit milliseconds (1×→100×)	INDICATIVE (mock, pure-Python)	"modeled / kept interactive as it scaled"
ClickHouse PoC: 8–130 ms pivots / ~160 ms writes / ~140–200× vs Postgres / 250M rows	PoC benchmark (requirements notebook)	"the ClickHouse PoC achieved ..." (a PoC result, not a prod SLA)
ClickHouse sub-second on 900M rows; 4.5M rows/s	CITED target (team benchmark)	"the design target / cited benchmark"
Cluster baselines: 8.5% failures (37/437); 0% reproducible; 1 config/plan; median job ≈20 s (370 runs); 1-in-5 finalized plans zero stores	REAL (FRD live-tenant audit)	"measured on live tenants"
Cluster targets: <1 h to plan; ≥20 configs; <2% failures; p95 <500 ms; ≥60% top-3 acceptance	FRD TARGET (committed, in build)	"the design targets we set / are building against"

Never present a mock latency as production throughput. When in doubt, name the harness. Also be ready to say clearly **what you personally built** vs. what you reverse-engineered/designed vs. what's team/prototype — interviewers respect the precision.

7.5 One-page résumé skeleton (fill with §2 bullets)

TARUN MITTAL — Senior Software Engineer (Python · AI/Agents · Data)

Summary: 2–3 lines — agentic systems + multi-engine data architecture; impact-first.

IMPACT ANALYTICS — Senior Software Engineer

Agentic Assort Planner — AI-assisted multi-agent retail planning platform

- C1 agentic cluster copilot: intent → agent explores ≥20 configs, 3 gates [AI · flagship]
- C2 dedicated ClickHouse read plane: probes 1–20s → p95 <500ms target [Data/AI-infra]
- A1 multi-agent decomposition + validated DAG (0.54→0.99 offline) [AI]
- D2 insert-only/versioned ClickHouse planning store (0 mutations) [Data]
- D1 three-tier data spine (BigQuery / Postgres / ClickHouse+DuckDB) [Data/Arch]
- P1 BigQuery cost ↓ up to ~280× (2.9 TiB→3.3 GiB); caught \$327/2-row leak [Perf]
- C4 reproducible clusterings (0%→100%) + pins + delegation-level autonomy [AI-governance]
- L1 reconciled DB verdict → 3-gate decision that de-risked migration [Leadership]

Tech: Python, FastAPI, asyncio · OpenAI/Gemini, LangGraph-style orchestration, RAG (pgvector, BM25+dense, Cohere rerank), text-to-SQL · BigQuery, PostgreSQL, ClickHouse, DuckDB, Redis · Rust/Axum (adjacent) · GCP (Cloud Tasks/Scheduler, GCS, Secret Manager)

(8 shown — trim to 5–8 per JD; C-bullets are the present-tense flagship, A/D the platform, P/L the

Sources (résumé best-practice research)

- [The XYZ Formula — Wonsulting](#)
- [XYZ Method Resume — Teal](#)
- [Senior Software Engineer Resume Examples — IGotAnOffer](#)
- [Software Engineer Resume in 2025 — NestCV](#)

Product/architecture facts are grounded in your own repos and study artifacts: `agentic_assort_multiagent/`, `db_architecture_poc/` (+ `results/*.json`), `docs/01-07`, `product_artifacts/`, the AssortSmart data deep-dive / performance-findings deliverables, and the Agentic Cluster FRD v1.8 (+ companion HLD/LLD) for the clustering copilot (§9).

8. Authoritative product specifics (NotebookLM — requirements notebook)

Source: NotebookLM "AssortSmart Agentic Store Clustering & Module Requirements" (117 sources) · verified July 2026

These are the **canonical product/engineering facts** straight from the requirements notebook — use them verbatim in interviews; they extend/confirm §3–§5. Where they differ from the offline PoC, prefer these for "what the product is," and keep the PoC framing (mock timings) for "what I personally ran."

8.1 Agents & orchestration

- **Five specialist agents:** `AssortmentDataAgent` , `ClusteringAgent` , `HindsightAgent` , `LinePlanAgent` , `RecommendationAgent` .
- **Orchestration:** `LangGraph` for multi-agent reasoning with **Planner → Executor → Solver** nodes, paired with **FastMCP (Model Context Protocol)** to **discover and execute tools safely**. (This is the production orchestration; the offline `agentic_assort_multiagent` PoC is a deterministic re-implementation of the same router/planner/executor shape.)
- **MCP tool layer** (`mcp-core`) is how agents call tools — worth naming: "agents don't hard-code integrations; they discover tools over MCP."

8.2 Governed autonomy (concrete thresholds)

- **High confidence 0.85–1.00** → auto-approve and proceed.
- **Medium 0.65–0.84** → proceed but **flag for planner review**.
- **Low 0.00–0.64** → **halt the pipeline and escalate** to a human planner.
- **HITL consent gate:** for highly consequential/destructive actions, the graph **interrupts** to request explicit confirmation.
- **Overrides:** require a **≥10-character reason code**, permanently written to the **audit trail**.

Interview gold: "Autonomy is a banded policy — ≥ 0.85 auto, $0.65\text{--}0.84$ act-and-flag, < 0.65 halt-and-escalate — plus a hard consent gate on destructive actions and a mandatory reason code on every override."

8.3 Tech stack (as stated)

- **Backend:** Python 3.11–3.13, **FastAPI**, **Uvicorn** (ASGI).
- **LLMs (multi-provider):** OpenAI **GPT-4.1** / **GPT-4o** / **o4-mini**, Google **Gemini**, Anthropic **Claude**, **xAI**.
- **Databases:** **BigQuery** (historical facts/cubes) · **PostgreSQL** (editable transactional store) · **DuckDB** (in-memory pivot accelerator) · **ClickHouse** (`ReplacingMergeTree` , insert-only versioned OLAP/planning).
- **Memory / vector:** **Redis**, **Neo4j** / **Memgraph** (graph), and vector stores **pgvector** / **Qdrant** / **Chroma** / **FAISS**.
- **Streaming/messaging + Apache:** **Socket.IO (WebSockets)** + **SSE** (real-time); **Google Cloud Tasks** + **GCP Pub/Sub** (async queues); **Apache Parquet** (storage format); **Apache Kafka** (`aiokafka` /

`confluent-kafka`) for **async embedding jobs**.

Correction to earlier "no Apache" caveat. The product **does** use Apache tech — **Kafka** (async embedding jobs) and **Parquet** (columnar storage/interchange). What's still true: the planning-store *ingestion path* is batch ELT (BigQuery cube/rollup → Postgres/ClickHouse), not a streaming pipeline into the grid.

8.4 Database model & ClickHouse (as stated)

- **Tenant unification:** a **generic core with "flex fields"** absorbs each retailer's dynamic hierarchy **without schema changes**; **tenant isolation resolves via environment variables to separate schemas**, with **shared master tables + extension hooks** for per-client customization. (This is the mechanism behind "same codebase/queries/procedures across tenants.")
- **ClickHouse engines:** `ReplacingMergeTree` (upserts/latest-wins), `AggregatingMergeTree` (rollups), `VersionedCollapsingMergeTree` (out-of-order CDC). **Partition** `PARTITION BY toYYYYMM(week_date)` ; `ORDER BY (measure, hierarchy...)` to exploit **sparse-index granule skipping**. **Production:** `ReplicatedReplacingMergeTree` + **sharding** (ClickHouse Keeper, or physical **L1-table sharding to eliminate hotspots**) and **Projections** for optimized rollups.
- **Edit / undo mechanics:** `ALTER UPDATE` mutations are **forbidden**; edits **insert new version rows** (ms-epoch); latest state read via `FINAL` or `argMax(value, version) ... GROUP BY key` . **Up to 10 undo steps are cached in Redis** before async DB commit; **version-diffs render as dual-value cells**.
- **Store-clustering model:** **K-Means**, run **twice independently** — **Performance (numeric)** and **Product/attribute (alphabetic)** — then **intersected into composite grades like "A1"**, stored in **EAV tables**.

8.5 PoC gating + benchmark numbers (as stated)

- **Gate:** sub-second UI responsiveness over 250M synthetic rows.
- **Result:** ClickHouse **meets it** — **8–130 ms filtered pivot reads**, **~160 ms flat-scale writes** — a **~140x–200x speedup** over the legacy Postgres flat schemas.

How to cite these numbers honestly. They are the **ClickHouse PoC benchmark documented in the requirements notebook** (250M synthetic rows). Say **"the ClickHouse PoC achieved 8–130 ms pivot reads and ~140–200x over the legacy Postgres flat schema on 250M rows"** — i.e. a *PoC benchmark result*, not a production SLA and not necessarily your personal measurement. It's stronger and safer than the offline `db_architecture_poc` mock ms; use these for the DB-performance story and keep `mutations_used = 0` as the correctness property you personally proved.

8.6 Résumé bullets these unlock (add to §2)

- "Orchestrated a 5-agent LangGraph pipeline (Planner→Executor→Solver) with a FastMCP tool layer, and a banded autonomy policy (auto ≥ 0.85 / act-and-flag 0.65–0.84 / halt < 0.65) plus a human-in-the-loop consent gate + audited reason codes."
- "Proved ClickHouse as an insert-only/versioned planning store — sub-second pivots over 250M rows (8–130 ms reads, ~160 ms writes), ~140–200x over the legacy Postgres flat schema — using ReplacingMergeTree + argMax(version), Redis-cached 10-step undo, and REPLACE PARTITION bulk re-seeds (zero mutations)."
- "Designed a tenant-generic 'flex-field' core (env-var schema isolation + shared masters + extension hooks) so one codebase serves every retailer's hierarchy with no per-client schema."
- "Implemented composite store grades (A1/A2/...) by intersecting independent K-Means runs on performance (numeric) and attribute (alphabetic) features, persisted via an EAV model."

9. The Cluster Recommendation Copilot — the module you are building right now

Source: Agentic Cluster FRD v1.8 (PM edition, 2026-07-06) + companion HLD/LLD · live tenant audits (kik, briscoes)

This is the **current, concrete build** — the strongest thing on the résumé because it is present-tense ownership: a full agentic module from measured problem → architecture → phased rollout. Everything below is what/why/how at interview depth. When asked *"what are you working on right now?"*, this chapter is the answer.

9.1 WHAT — in one breath

"A conversational + structured copilot for store clustering. The planner states *what* to cluster and *for when* — a dedicated agent handles store scoping, attribute selection, exploration and scoring on a dedicated ClickHouse read plane; the planner reviews evidence, adjusts stores with consequences shown, and approves. The existing product consumes the result completely unchanged."

Two entry modes, one agent: the **manual wizard with inline recommendations** (Mode A) and **chat-native** (Mode B — "cluster Small Appliances for SS27"). The **convergence rule** is non-negotiable: whichever door you enter, the flow converges before execution — same config document, same search plan, same three gates, same write-back. A session started in one mode can continue in the other; the modes are two surfaces on one agent, never two implementations.

9.2 WHY — the measured problem (all live-audit numbers)

Store clustering is the foundation every strategy plan is built on — and the least assisted step in the product:

Measured reality (today)	Number	Source
Configurations evaluated per plan	exactly 1 (search space: 12 algorithms × k 3–10 × any attribute subset)	live audit
Run failure rate	8.5% (37/437 runs, kik) — >80% are input mistakes at the data boundary	live audit
Reproducibility of shipped clusterings	0% — algorithm/params/seed never persisted	live audit
Manual store swaps surviving a re-run	0% — silently erased	live audit
Significance scores (the hardest signal)	computed, then ignored — user picks from raw score lists	code + UX audit
Median clustering job	≈20 s (370 live runs; max 463 s)	live audit
Agent data-probe latency on shared BQ	1–20 s+, variance uncontrolled	live audit
Strategy-create help choosing a cluster plan	zero — and 1 in 5 finalized kik plans has zero stores attached ; briscoes' finalized inventory is 100% >17 months stale	live audit

The one-line diagnosis for interviews: **"the compute is fast (~20 s median); the bottleneck is everything around the machine"** — expert choices made blind up-front, one shot per try, knowledge thrown away after.

9.3 HOW — the workflow inversion

Today's flow asks the human for four expert choices up front, blind, then runs once. The agentic flow inverts it:

```
TODAY:  human picks hierarchy → period → store groups → attributes → submit(1 config) → eyeball
AGENTIC: human states INTENT (hierarchy + period — the only 2 non-derivable inputs)
        → agent proposes store scope      (active stores, sister-store subs, sample-size gua
        → agent proposes attribute sets    (significance + coverage + redundancy gates, ranke
        → agent explores 20–100 pruned configs on an isolated scratch plan (results stream)
        → top-3 with evidence packs (driving attributes, separation, stability, size balance,
          churn-vs-current-live-clusters)
        → human reviews, what-ifs (≤10 s warm), moves/pins stores with impact shown
        → explicit approval → write-back into today's tables
```

Key definitions you must own in the room:

- **Intent** = hierarchy scope + reference period — the only two inputs that are business strategy rather than derivable from data. Everything else (store cohort, attributes, algorithm, k, tuning) the agent derives. Pins/moves are *not* intent — they're **business-knowledge injection** during review. The invariant: *intent is always human-owned; the agent may ground, expand, execute or queue it — never invent or auto-resolve one.*
- **Config document** = the unit of record: algorithm, k, hyperparameters, seed, features, windows, pins, data watermark, scorer version — persisted and **content-addressed**. "Re-run" reproduces membership on the same watermark or explains drift.
- **Pins** = durable constraints with reason codes; honored by every subsequent re-run and refresh. This kills the silent-loss-of-swaps failure (0% → 100% survival).
- **Three delegation levels: L1 copilot** (proposes every step, executes on confirmation — launch) → **L2 autopilot-with-approval** (one intent runs the whole flow unattended; one review, one approval) → **L3 standing monitor** (nightly drift detection — "12 stores no longer fit, stability dropped 0.18" — raises a proposal). Nothing at any level auto-finalizes; system-originated work surfaces as **pending intents** unless the tenant grants a recorded, one-time standing pre-approval.

9.4 The architecture positions (and why)

1. **Deterministic tools compute; the LLM orchestrates and explains.** Every number in an evidence pack traces to a tool call; same inputs + same scorer version ⇒ identical ranking. The LLM never invents a metric. This is the anti-hallucination stance *and* what makes recommendations auditable.
2. **Deterministic grounding for chat.** Department mentions are backtracked to full hierarchy paths via catalog search (ambiguity → clarifying question, never a guess); season labels resolve via the tenant's fiscal calendar (seasons differ per tenant and hemisphere); the **grounding card** (scope, cohort, attributes, each with a reason) must be confirmed before any compute.

3. **A dedicated ClickHouse read plane.** Agent probes need p95 < 500 ms deterministic (vs 1–20 s+ on shared BigQuery slots). Fed by the org's existing BQ→CH ingestion pipeline (proven on ItemSmart); **agent access is read-only enforced by database profile, not convention**; exploration writes only to isolated scratch plans; nightly precompute makes active hierarchies feel instant (≥80% cold→warm target). This is the gated ClickHouse adoption from the reconciled DB verdict actually happening — read plane only; BigQuery stays truth; Postgres stays the editable SoR (explicit FRD non-goal).
4. **Write-back compatibility.** Approved results land in the existing `plan_cluster_final` / `plan_cluster_store_final` tables — strategy, budget, wedge, line consume them with **zero product changes**. The legacy wizard keeps working untouched in parallel (adoption de-risk).
5. **Safe editing of consumed plans.** Editing a plan referenced by strategies defaults to **clone-and-version with lineage** (in-place edits would silently invalidate downstream budgets/wedges); rebinding is explicit per-strategy; an edit-impact card shows consumers *before* any change.
6. **Every flow closes.** A 19-workflow audit produced a **terminal-states contract**: every flow defines success / rejection / failure / abandonment-expiry. Degraded batches (<3 viable candidates) surface partials + a failure taxonomy + retry; capacity is a queue with a position banner, not an error; rejection returns to comparison and is captured as a learning event.
7. **Honest learning loops.** Deviations (attribute removed, store moved, non-top pick, dismissal) are captured with reason chips: (a) deterministic memory — never blindly re-recommend what this scope rejected; (b) human-approved calibration of tenant metric weights (versioned, never silently self-tuned); (c) a labeled recommended-vs-chosen corpus for any future learned ranker, shipped only as an explicit scorer-version release to preserve determinism.

9.5 The strategy doorway (§2.2 of the FRD — best cross-module story)

Every strategy plan binds to exactly one cluster plan at creation; today there's zero help and the audit found strategies pointing at cluster plans that **no longer exist**. The fix, three tiers:

- **Hard eligibility gate before scoring:** finalized $\wedge$ not deleted $\wedge$ non-empty membership $\wedge$ recoverable scope. (1 in 5 finalized kik plans has zero stores — binding one silently breaks every downstream step. Unusable plans are listed, never scored.)
- **FitScore ≤ 2 s, deterministic:** cheap terms rank all candidates; the expensive variance-explained term ("these clusters explain X% of store-revenue variance in your scope") runs top-5-only, computed on the **last comparable season** (a future window has no sales yet), **penalized for cluster count** (so many-tiny-clusters can't game it) and **weighted by store coverage**. Bands Green/Yellow/Red **calibrated by backtesting against 467 historical bindings** (363 kik + 104 briscoes).
- **Always a top-3 shortlist** with explain cards + pairwise deltas ("#2 covers your scope but is 9 months staler"); padded with a **spawn card** when fewer exist — never an empty dropdown. Spawn runs the full recommendation pipeline inline, writes at the pre-finalize step, promotes only on human confirmation (or parks as a **pending intent** where tenant governance requires a cluster-owner sign-off), records

`created_via` + full scope keys for future dedup, and the create API now validates bindings are live + finalized — eliminating the dangling-reference class.

9.6 Targets & rollout (know these cold)

Metric	Baseline (measured)	Target (6 mo)
Hierarchy → finalized plan	hours–days	< 1 h (<10 min precomputed)
Configs evaluated per plan	1	≥ 20
Run failure rate	8.5%	< 2%
Top-3 contains shipped config	n/a	≥ 60%
Store corrections surviving re-run	0%	100%
Agent probe latency	1–20 s+	p95 < 500 ms
Reproducible shipped clusterings	0%	100%
Strategy plans using a suggested cluster	0%	≥ 40%

Phases: P0 read-only copilot (CH instance + cubes + significance/explain tools; exit: p95 <500 ms, cube reconciles to BQ within 0.1%) → P1 recommendation engine (batch exploration + what-if + config documents; exit: top-3 ≤60 s cold, deterministic re-rank) → P2 approve & write-back (gates, pins with impact preview, doorway point-at-existing; exit: end-to-end consumption in staging) → P3 scale-out (nightly precompute, L2/L3, spawn, kik at 6,040 stores — gated on kik's BQ env cutover).

Top risks + mitigations (one-liners): derived-data rot → owned ingestion + freshness sentinel + data-as-of quoted in every rec; metric gaming → hard gates before scoring, stability as pass/fail, full metric vector always shown; trust gap → evidence packs + churn-vs-current + P0 ships read-only value first; compute tail (463 s max) → async batches with streamed partials + nightly precompute for heavy hierarchies.

9.7 Interview Q&A for this module

Q: Why an agent at all — isn't this just AutoML for clustering? A: AutoML optimizes a loss; this optimizes a *decision* a human must own. The agent's job is search + evidence + explanation; the human's job is intent, business-knowledge injection (pins), and accountability (`is_final` is a human signature). The measured problem wasn't model quality — it was 1 config tried per plan, 8.5% avoidable failures, zero reproducibility. Those are workflow failures, so the fix is a workflow agent with hard governance, not a better optimizer.

Q: How do you stop the LLM from hallucinating a recommendation? A: It structurally can't — deterministic tools compute every number, the LLM only orchestrates and narrates, and every figure in an evidence pack

traces to a tool call. Scope grounding is catalog search, season resolution is the fiscal calendar, and ranking is a versioned deterministic scorer: same inputs $\Rightarrow$ identical order.

Q: Why ClickHouse here when the earlier audit said no? A: The audit's "no" was about the *legacy editable* workload (in-place OLTP mutations). This is the opposite shape: a **read-only** analytical plane for agent probes, insert-only ingestion from BigQuery, isolated from the product's write path. It's exactly the gated forward option my reconciled verdict defined — and the FRD non-goals pin it: no transactional data moves, BigQuery stays truth.

Q: What happens when the agent is wrong or the batch degrades? A: Defined outcomes everywhere (the terminal-states contract): <3 viable candidates $\rightarrow$ partial results + failure taxonomy + one-tap retry; autopilot ambiguity $\rightarrow$ parked as a pending intent, never auto-resolved; capacity $\rightarrow$ queue with a position banner; rejection $\rightarrow$ back to comparison, reason captured as a learning event. Nothing accumulates silently; sessions expire at the tenant retention boundary.

Q: How does it learn without becoming non-deterministic? A: Three honest loops — deterministic memory (never re-recommend what this scope rejected, acknowledged in-session), human-approved calibration (aggregated deviations inform the next *versioned* weight set), and a labeled corpus for a future ranker shipped only as an explicit scorer-version release. No silent self-tuning, ever.

Q: Multi-tenancy? A: Per-tenant everything: ingestion lanes, quotas/concurrency budgets, k-ranges and algorithm allow-lists, metric weights (versioned, actually consumed — fixing a known ignored-weightage defect), default-hierarchy registries with draft vs standing-pre-approval governance, and ClickHouse placement per tenant (briscoes = AU, kik = EU) as a config decision, not a redesign. kik launch gates on its BigQuery environment cutover — its prod dataset is an empty shell today.

Q: What did you deliberately not build? A: Four explicit non-goals: no auto-finalize without human approval; no replacement of the existing wizard (parallel, untouched); no transactional/plan data on ClickHouse and no replacing BigQuery as truth; no attempt to make the ML itself faster (~ 9 s/config median is accepted physics — we parallelize and precompute around it).